



Citre

Competition

October 21, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	High Risk	4
3.1.1	Unsafe UTXO Selection Allows DoS via RBF Attack on DA Service	4
3.1.2	DoS Vulnerability in <code>send_raw_deposit_transaction</code> Allowing Unlimited Deposit Mem- pool Growth	5
3.2	Medium Risk	7
3.2.1	SPV Merkle Proof Duplication Allows Forged Transaction Inclusion	7
3.2.2	Infinite timeout in the Bitcoin JSON-RPC client can freeze all critical Citrea compoments	9
3.2.3	Logic Error in Reorg Detection in chain-state update triggers unnecessary reorg handling	10
3.2.4	Insufficient Funds Due to Incorrect Amount Comparison	13
3.2.5	Infinite HTTP client timeout in FeeService can freeze the sequencer	13
3.2.6	Off-by-one error in <code>get_pending_commitments</code> causes chain halt	15
3.2.7	Bitcoin DA Unbounded pre-decompression accumulation causes DoS/OOM	17
3.2.8	Unauthenticated DoS via JsTracer on <code>debug_traceBlockByNumber</code> Endpoint	19
3.2.9	Failed deposit transactions cause nonce mismatch and break L2 block production . .	21
3.2.10	Proof-Slot Leak Halts Parallel Prover	23

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A competition provides a broad evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While competitions endeavor to identify and disclose all potential security issues, they cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities, therefore, any changes made to the code would require an additional security review. Please be advised that competitions are not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above matrix. High severity findings represent the most critical issues that must be addressed immediately, as they either have high impact and high likelihood of occurrence, or medium impact with high likelihood.

Medium severity findings represent issues that, while not immediately critical, still pose significant risks and should be addressed promptly. These typically involve scenarios with medium impact and medium likelihood, or high impact with low likelihood.

Low severity findings represent issues that, while not posing immediate threats, could potentially cause problems in specific scenarios. These typically involve medium impact with low likelihood, or low impact with medium likelihood.

Lastly, some findings might represent improvements that don't directly impact security but could enhance the codebase's quality, readability, or efficiency (Gas and Informational findings).

2 Security Review Summary

Citrea is the first rollup that enhances the capabilities of Bitcoin blockspace with zero-knowledge technology, making it possible to build everything on Bitcoin.

From Jul 18th to Aug 15th Cantina hosted a competition based on [Citrea](#). The participants identified a total of **84** issues in the following risk categories:

- High Risk: 2
- Medium Risk: 10
- Low Risk: 38
- Gas Optimizations: 0
- Informational: 34

The present report only outlines the **high** and **medium** risk issues.

3 Findings

3.1 High Risk

3.1.1 Unsafe UTXO Selection Allows DoS via RBF Attack on DA Service

Submitted by [blockace](#), also found by [franfran](#) and [cdpandora](#)

Severity: High Risk

Context: (No context files were provided by the reviewer)

Summary: The `get_utxos` function in the Bitcoin DA service filters UTXOs based on `spendable`, `solvable`, and `amount` criteria but fails to check the `safe` field. This allows the service to select unconfirmed UTXOs (those sitting in the mempool) as inputs for commitment transactions. An attacker can exploit this by submitting Replace-By-Fee (RBF) transactions to cause DA service transaction failures.

Finding Description: Vulnerable Code Location: `crates/bitcoin-da/src/service.rs:346-369`:

```
pub(crate) async fn get_utxos(&self) -> Result<Vec<UTXO>> {
    let utxos = self
        .client
        .list_unspent(Some(0), None, None, None, None)
        .await?;
    // ...
    let utxos: Vec<UTXO> = utxos
        .into_iter()
        .filter(|utxo| {
            utxo.spendable           // Checked
            && utxo.solvable         // Checked
            && utxo.amount > Amount::from_sat(REVEAL_OUTPUT_AMOUNT) // Checked
            // MISSING: utxo utxo.safe
        })
        .map(Into::into)
        .collect();
    // ...
}
```

Attack Flow:

1. Setup Phase: Attacker identifies the DA service's wallet address.
2. Bait Transaction: Attacker sends a BTC transfer transaction to the DA wallet with:
 - RBF (Replace-By-Fee) enabled.
 - Very low fee rate to keep it unconfirmed in mempool.
 - Sufficient amount to pass the `REVEAL_OUTPUT_AMOUNT` filter.
3. UTXO Selection: When DA service calls `get_utxos()`, the function includes the attacker's unconfirmed UTXO because:
 - `spendable = true` (wallet can spend it).
 - `solvable = true` (wallet has keys).
 - `safe = false` (unconfirmed) ← NOT CHECKED.
4. Transaction Building: DA service uses the unsafe UTXO as input in `create_da_transactions_with_fee_rate()`.
5. RBF Attack: Attacker replaces the original transaction with a different one (e.g., sending funds elsewhere).
6. Transaction Failure: DA service's commitment transaction fails because it references a UTXO that no longer exists.

Impact Explanation: Critical DA operations (ZK proof submissions, sequencer commitments, batch proof method IDs) can be disrupted, causing DoS.

Likelihood Explanation: The attack requires minimal Bitcoin (just enough to exceed `REVEAL_OUTPUT_AMOUNT`), and is easy to execute.

Recommendation: Add safe field validation to the UTXO filter:

```
pub(crate) async fn get_utxos(&self) -> Result<Vec<UTXO>> {
    let utxos = self
        .client
        .list_unspent(Some(0), None, None, None, None)
        .await?;
    if utxos.is_empty() {
        return Err(BitcoinServiceError::MissingUTXO);
    }

    let utxos: Vec<UTXO> = utxos
        .into_iter()
        .filter(|utxo| {
            utxo.spendable
            && utxo.solvable
            && utxo.safe // ← ADD THIS CHECK
            && utxo.amount > Amount::from_sat(REVEAL_OUTPUT_AMOUNT)
        })
        .map(Into::into)
        .collect();
    if utxos.is_empty() {
        return Err(BitcoinServiceError::MissingSpendableUTXO);
    }

    Ok(utxos)
}
```

As an additional Recommendation, consider requiring minimum confirmations (e.g., 1-3 blocks) by modifying the `list_unspent` call:

```
let utxos = self
    .client
    .list_unspent(Some(1), None, None, None, None) // Require at least 1 confirmation
    .await?;
```

Chainway Labs: Fixed in citrea PRs 2715 and 2867.

Jakub: Fix verified.

3.1.2 DoS Vulnerability in `send_raw_deposit_transaction` Allowing Unlimited Deposit Mempool Growth

Submitted by [lian886](#), also found by [TheStryke](#), [marchev](#) and [TamayoNft](#)

Severity: High Risk

Context: [rpc.rs#L276](#)

Summary: The `send_raw_deposit_transaction` RPC endpoint allows attackers to repeatedly submit identical deposit transaction data, causing unlimited growth of the deposit mempool. This can lead to memory exhaustion and significant delays in processing legitimate deposit transactions due to the limited processing capacity per block.

Finding Description: In the sequencer's RPC implementation (`crates/sequencer/src/rpc.rs`), the `send_raw_deposit_transaction` function accepts deposit transaction data without any duplicate detection mechanism:

```
fn send_raw_deposit_transaction(&self, deposit: Bytes) -> RpcResult<> {
    debug!("Sequencer: citrea_sendRawDepositTransaction");

    let evm = Evm::<DefaultContext>::default();
    let mut working_set = WorkingSet::new(self.context.storage.clone());

    let dep_tx = self
        .context
        .deposit_mempool
        .lock()
        .make_deposit_tx_from_data(deposit.clone().into());

    let tx_res = evm.get_call( // <<<
        dep_tx,
        Some(BlockId::pending()),
```

```

        None,
        None,
        &mut working_set,
        &self.context.ledger,
    );

    match tx_res {
        Ok(hex_res) => {
            tracing::debug!("Deposit tx processed successfully {}", hex_res);
            self.context
                .deposit_mempool
                .lock()
                .add_deposit_tx(deposit.to_vec()); // <<<
            Ok(())
        }
        Err(e) => {
            error!("Error processing deposit tx: {:?}", e);
            Err(e)
        }
    }
}

```

A malicious actor can exploit this by submitting the same deposit transaction multiple times in a short period. This function uses `evm.get_call()` to validate whether the deposit transaction would succeed before it is added to the `deposit_mempool`. Since both submissions are validated against the same EVM state from the latest block, they are both seen as valid and added to the `deposit_mempool`. However, `deposit_mempool` is essentially just a `VecDeque` with no checking mechanisms:

```

pub struct DepositDataMempool {
    /// Queue of accepted deposit transaction data
    accepted_deposit_txs: VecDeque<Vec<u8>>,
}

```

This has three critical security issues:

1. It lacks any authentication or rate limiting mechanisms.
2. It does not check if the same deposit data has been previously submitted.
3. There is no upper bound on the size of the deposit mempool.

The vulnerability is exacerbated by the `deposit_mempool_fetch_limit` configuration (typically set to 10), which limits the number of deposit transactions processed per block. This creates a situation where an attacker can flood the `deposit_mempool` with duplicate transactions that will take an extensive time to clear.

Impact Explanation: The impact of this vulnerability is High due to two severe consequences:

1. **Memory Resource Exhaustion:** With no upper bound on the deposit mempool size, an attacker can cause continuous memory growth by submitting the same deposit data repeatedly. This can eventually lead to node crashes, service disruptions, or system instability.
2. **Effective DoS on Cross-Chain Operations:** Due to the `deposit_mempool_fetch_limit` (typically 10 transactions per block), legitimate deposit transactions can be severely delayed. For example, if an attacker adds 100,000 duplicate transactions to the mempool, new legitimate deposit transactions would be delayed by approximately 10,000 blocks. This effectively renders the cross-chain bridge functionality unusable during this period.

Likelihood Explanation: The likelihood of this vulnerability being exploited is Medium because:

1. No Bitcoin funds or special permissions are required to execute this attack.
2. The attack can be executed remotely by anyone with RPC access to a sequencer node.

Proof of Concept: An attacker could exploit this vulnerability with the following steps:

1. Obtain normal deposit transaction data from the Bitcoin chain and continuously submit it through the `send_raw_deposit_transaction` RPC endpoint.
2. The sequencer would process each submission, performing resource-intensive validation for each duplicate.
3. All duplicates would be added to the deposit mempool.

4. Memory usage would continually increase as duplicates are continuously added, potentially causing the sequencer node to crash due to insufficient memory.
5. With the default `deposit_mempool_fetch_limit` of 10, it would take a long time to process these duplicates, preventing other deposit transactions from being processed.

Recommendation: Add a check to verify whether the deposit transaction already exists in `deposit_mempool`. When a deposit transaction already exists in `deposit_mempool`, the RPC should directly return an error.

Chainway Labs: Fixed in citrea PRs [2692](#) and [2891](#).

Jakub: Fix verified.

3.2 Medium Risk

3.2.1 SPV Merkle Proof Duplication Allows Forged Transaction Inclusion

Submitted by [Codertjay](#), also found by [jeanpierrepolnaref](#)

Severity: Medium Risk

Context: [BitcoinLightClient.sol#L110-L114](#)

Summary: The `BitcoinLightClient` contract, which is responsible for verifying that transactions have been included in the Bitcoin blockchain, contains a critical flaw. A malicious user can take a valid proof for one transaction and use it to trick the contract into believing a completely different, non-existent transaction is also valid. This vulnerability allows an attacker to forge deposit transactions and directly steal funds from the Citrea bridge, compromising the core security of the rollup.

Finding Description: Imagine a courtroom where a piece of evidence (a photo) is presented. This photo is our Bitcoin transaction (`_txId`). The photo is stored in a sealed, tamper-proof evidence bag, which is our Merkle Root (`_merkleRoot`).

To prove the photo is authentic, a witness provides a list of chain-of-custody receipts (`_proof`). The witness also states on the record, "This photo is Exhibit #1" (our `_index`).

The judge (the `BitcoinLightClient` contract) diligently checks the receipts. The receipts correctly prove that the photo is from the official evidence bag. The problem is that the judge never checks if "Exhibit #1" is a valid exhibit number for this case.

An attacker can exploit this. They can take the same photo and the same valid receipts, but lie to the judge and say, "This photo is Exhibit #5" (an invalid number for a case with only 4 exhibits). Because the judge only checks the receipts and not the exhibit number, they are fooled and accept the photo as valid evidence for Exhibit #5.

This is the flaw: one piece of real evidence can be used to validate multiple claims, which should be impossible.

Technical Details: The `verifyInclusion` function in `BitcoinLightClient.sol` delegates Merkle proof verification to the `ValidateSPV.prove` library. This library correctly validates the cryptographic path of the proof. However, it fails to validate that the provided `_index` is within the valid range for the tree's size.

A Merkle tree of depth `h` (which is determined by `_proof.length`) can only contain 2^{**h} leaves. Therefore, valid indices must be in the range $0 \leq \text{_index} < 2^{**h}$. The `ValidateSPV.prove` function is missing this crucial `require(_index < 2^{**h})` check.

Because the verification algorithm uses the individual bits of `_index` to navigate the tree path, an out-of-bounds index like 1 and an invalid index like 5 (which is $1 + 2^{**2}$) will have the same lower bits. The flawed verifier only looks at these lower bits, calculates the same path, and incorrectly returns `true` for both.

Impact: This vulnerability is of High impact as it leads to a complete breakdown of the bridge's security model.

1. **Direct Theft of Funds:** An attacker can execute a "phantom deposit." They can find a small, real deposit from another user, get its valid proof, and then submit a transaction to the bridge claiming they made a massive deposit. They present their fake transaction ID but use the real proof and a manipulated index. The contract will be tricked into verifying the fake deposit, minting unbacked assets for the attacker which can be used to drain all liquidity from the bridge.

2. State Corruption & Loss of Trust: The core promise of the rollup is that its state is secured by the Bitcoin L1. This vulnerability breaks that promise. If proofs of inclusion are meaningless, the entire system is untrustworthy.

Proof of Concept: The following test, when added to `BitcoinLightClient.t.sol`, provides definitive proof of the vulnerability. The test passes because the second `assertTrue` call, which should fail, incorrectly succeeds:

```
// Add this test to
↪ [BitcoinLightClient.t.sol](../crates/evm/src/evm/system_contracts/test/BitcoinLightClient.t.sol)
// This test was created and validated during the audit.
function test_Vulnerability_MerkleProofDuplication() public {
    // 1. Use the known-good data from the existing `testVerifyInclusion` test.
    bitcoinLightClient.initializeBlockNumber(INITIAL_BLOCK_NUMBER);
    bytes32 root = hex"DBEE9A868A8CAA2A1DDF683AF1642A88DFB7AC7CE3ECB5D043586811A41FDBF2";
    uint256 treeDepth = 2;
    bitcoinLightClient.setBlockInfo(mockBlockHash, root, treeDepth);

    bytes32[] memory proof = new bytes32[](2);
    proof[0] = hex"0000000000000000000000000000000000000000000000000000000000000000";
    proof[1] = hex"6B1DAB5721B7B8D68B2C7F795D689998A35EFED7E5C99E12E6C8D5C587A1628D";
    bytes32 wtxId = hex"A28E549DC50610430BF7E224EFFD50DB0662356780C934AF0F1A9EB346D50087";

    // 2. VERIFY: The proof is valid for the CORRECT index.
    uint256 correctIndex = 1;
    assertTrue(
        bitcoinLightClient.verifyInclusion(mockBlockHash, wtxId, abi.encodePacked(proof), correctIndex),
        "Proof should be valid for correct index"
    );

    // 3. THE ATTACK: Use the SAME proof for a malicious, out-of-bounds index.
    // The tree size is 2**depth = 4. The malicious index is correctIndex + treeSize.
    uint256 maliciousIndex = correctIndex + (1 << treeDepth); // 1 + 4 = 5

    // 4. VERIFY: The contract incorrectly accepts the proof for the WRONG index.
    // This should fail, but it passes, proving the vulnerability.
    assertTrue(
        bitcoinLightClient.verifyInclusion(mockBlockHash, wtxId, abi.encodePacked(proof), maliciousIndex),
        "VULNERABILITY: Proof was accepted for a malicious, out-of-bounds index"
    );
}
```

The foundry shell:

```
system_contracts git:(nightly) forge test --mt test_Vulnerability_MerkleProofDuplication -vvv
[] Compiling...
[] Compiling 1 files with Solc 0.8.30
[] Solc 0.8.30 finished in 3.37s
Compiler run successful!

Ran 1 test for test/BitcoinLightClient.t.sol:BitcoinLightClientTest
[PASS] test_Vulnerability_MerkleProofDuplication() (gas: 121527)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.27ms (1.02ms CPU time)

Ran 1 test suite in 2.25s (3.27ms CPU time): 1 tests passed, 0 failed, 0 skipped (1 total tests)
system_contracts git:(nightly)
```

Recommendation: The vulnerability must be fixed immediately by adding a bounds check in the `BitcoinLightClient` contract before it calls the external library. This ensures that even if the library is flawed, the Citrea protocol is not vulnerable:

```
function _verifyInclusion(bytes32 _blockHash, bytes32 _wtxId, bytes calldata _proof, uint256 _index) internal
↪ view returns (bool) {
    uint256 treeDepth = coinbaseDepths[_blockHash];
    require(_proof.length == treeDepth * 32, "Invalid proof length");

    // FIX: Add index bounds validation
    require(_index < (1 << treeDepth), "Index out of bounds");

    bytes32 _witnessRoot = witnessRoots[_blockHash];
    return ValidateSPV.prove(_wtxId, _witnessRoot, _proof, _index);
}
```

The same fix should be applied to `verifyInclusionByTxId`. This ensures that each Merkle proof can only validate its intended leaf, securing the bridge from this attack vector.

Chainway Labs: Fixed in [citrea PR 2870](#) and [bitcoin-spv PR 2](#).

Jakub: Fix verified.

3.2.2 Infinite timeout in the Bitcoin JSON-RPC client can freeze all critical Citrea components

Submitted by [marchev](#)

Severity: Medium Risk

Context: [service.rs#L119](#)

Summary: Citrea uses a forked version of the `rust-bitcoincore-rpc` library maintained by its own development team. While the original crate uses a transport layer with a default 15-second timeout, Citrea's fork replaces it with the `request` HTTP client without setting any timeouts. By default `request` is configured with an infinite timeout. As a result, any Bitcoin Core JSON-RPC call can hang indefinitely if the network becomes unresponsive. This affects all major Citrea components (incl. the sequencer, full node, batch prover and light client) since they all rely on the DA layer which internally uses this forked client.

Finding Description: The `BitcoinService` is the central component responsible for interacting with a Bitcoin Core node in the Citrea architecture. It handles operations like sending commitments, retrieving headers, monitoring transaction states, estimating fees, etc. All major Citrea components (sequencer, full node, batch prover, and light client) rely on the DA service.

`BitcoinService` uses the `rust-bitcoincore-rpc` library to interface with a Bitcoin Core node over JSON-RPC. However, instead of the original upstream crate ([rust-bitcoin/rust-bitcoincore-rpc](#)), it uses a fork of the library ([chainwayxyz/rust-bitcoincore-rpc](#)) maintained by Chainway Labs (the Citrea dev team).

The original crate uses `apoelstra/rust-jsonrpc`'s `simple_http` transport layer, which enforces a default timeout:

```
const DEFAULT_TIMEOUT: Duration = Duration::from_secs(15);
```

This ensures that hung requests fail after a reasonable period.

In contrast, Chainway's fork replaces the transport layer via [PR 5](#) to use `request`, a more flexible and modern HTTP client:

```
impl RequestTransport {
    pub fn new(url: Url) -> Self {
        Self {
            client: request::Client::new(), // infinite timeouts by default
            url,
        }
    }
    // ...
}
```

However, `request::Client::new()` applies [no timeouts](#) unless timeouts are configured via the builder. This results in infinite read and total timeout for requests:

```
ClientBuilder {
    config: Config {
        read_timeout: None, // Infinite read timeout
        timeout: None,     // Infinite total timeout
        // ...
    }
}
```

Internally, timeouts are [only applied if these fields are populated](#). When both are `None`, no timeout logic is triggered and the request hangs indefinitely if the connection stalls:

```
let total_timeout = timeout
    .or(self.inner.request_timeout)
    .map(tokio::time::sleep) // None if None
    .map(Box::pin);

let read_timeout_fut = self.inner.read_timeout
```

```
.map(tokio::time::sleep) // None if None
.map(Box::pin);
```

This is problematic because this RPC client is used for all communication with Bitcoin Core. With no timeout, any temporary network issue can cause any critical Citrea component (such as the sequencer, full node, batch prover or light client) to freeze virtually infinitely since all RPC calls in the `BitcoinService` are awaited.

Note that while the `BitcoinService` does use exponential backoff for RPC calls, this does not mitigate the issue. Backoff logic only activates when a request returns with an error. In the case of a stalled or indefinitely hanging request, no error is returned and backoff never triggers.

Impact Explanation: The bug compromises system-wide liveness. All Bitcoin Core communication is centralized in the DA service, which uses an HTTP client with no timeouts. Any temporary network issue (packet loss, blackhole, routing error) during a Bitcoin JSON-RPC can indefinitely block any Citrea component.

Likelihood Explanation: The scenario does not rely on edge conditions. Any of the following can trigger it:

- Common network instability (e.g. partial disconnect, unacknowledged packet loss).
- Misconfigured infrastructure (e.g. routing loop).
- Malicious routing (e.g. BGP hijack).

Because all DA logic routes through the same infinite-timeout RPC client, this is a single point of systemic failure.

Proof of Concept:

1. Run the Citrea sequencer locally.
2. Drop traffic to the Bitcoin Core node:

```
sudo iptables -A OUTPUT -d <bitcoin-node-ip> -j DROP
```

3. Trigger or wait for any DA logic.
4. Observe the hang.

Recommendation: Consider using the original `rust-bitcoincore-rpc` crate, which applies a sensible default timeout and avoids this issue entirely.

Alternatively, if continuing to use the custom fork, add timeout configuration for the `request` transport layer as follows:

```
let client = request::Client::builder()
    .timeout(Duration::from_secs(5))
    .build()?;
```

Chainway Labs: Fixed in [citrea PR 2898](#).

Jakub: Fix verified.

3.2.3 Logic Error in Reorg Detection in chain-state update triggers unnecessary reorg handling

Submitted by [uint256viet](#), also found by [0xgh0st](#), [0xAlex](#) and [Petrus](#)

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Finding Description: The `check_chain_state()` does those following things:

1. Only proceeds if the blockchain tip has changed since last check. if `new_tip != chain_state.current_tip` {.
2. Goes backwards from the new tip, collecting block hashes for the last `finality_depth` blocks.
Go from `i=1` and start with `current_hash` of new block height -1.

```

for i in 1..=self.finality_depth {
    //@audit Goes backwards from the new tip, collecting block hashes for the last finality_depth blocks.
    let height = new_height.saturating_sub(i);
    current_hash = self.client.get_block_hash(height).await?;
    new_blocks.push((current_hash, height));
}

```

3. It checks if any of the new blocks match blocks from the previously stored chain. If a match is found but at a different position, it means the chain reorganized.

```

if let Some(pos) = chain_state
    .recent_blocks
    .iter()
    .position(|&(hash, _)| hash == current_hash)
{
    if pos != i as usize {
        reorg_detected = true;
        reorg_depth = i;
    }
    break;
}

```

4. The things here: They compare pos(the position found the hash == current_hash) so pos is an index in the old recent_blocks array start with 0 while i(the depth from the current new tip start with 1) => These aren't directly comparable.

Ex: Given the following recent_blocks:[100(A), 99(B), 98(C), 97(D)].

- When a new tip 101(X) arrives, the loop with i = 1 => it calculate current hash of height-1(101-1) aka block100(A).
- It then compares block 100(A) against the element at position 0 in the recent_blocks list which is 100(A).
- This always matched (Hash matched && pos!=i: 0!=1 matched).
- => reorg_detected = true incorrectly because the check relies on array position rather than depth alignment.

Impact Explanation: Medium.

- Unwarranted reorg handling on each block, inflating work every interval.
- Increased RPC pressure (get_block_hash, get_transaction, etc.)=> because each reorg handle will iter through all monitor transaction degrading performance.
- Misleading logs/metrics and redundant rebroadcast checks for near-tip transactions.

Likelihood Explanation: High. Always reorg detected.

Proof of Concept:

1. Apply this log for more info:

```

if pos != i as usize {
    println!("-----reorg_detected");
    reorg_detected = true;
    reorg_depth = i;
}

```

2. Start bitcoin regtest and sequencer with the docs/run-dev.md.

- Build bitcoin binary.
- Run bitcoin testnet on docker: docker compose -f docker/docker-compose.regtest.yml up.
- Setup sequencer wallet and config.
- bitcoin-cli -regtest -rpcuser=citrea -rpcpassword=citrea createwallet citreatesting.
- bitcoin-cli -regtest -rpcuser=citrea -rpcpassword=citrea loadwallet citreatesting.
- Changes config: `sequencer_rollup_config.toml`

```
[da]
# fill here
node_url = "http://127.0.0.1:18443"
# fill here
node_username = "citrea"
node_password = "citrea"
```

3. Mine some bitcoin blocks for the wallet generated above utxo: `bitcoin-cli -regtest -rpcuser=citrea -rpcpassword=citrea -generate 201`.
4. Edit config for faster check reorg, currently is 60s:

```
pub const fn check_interval() -> u64 {
    1 //change from 60s to 1s
}
```

5. Start sequencer:

```
./target/debug/citrea --dev --da-layer bitcoin --rollup-config-path
→ resources/configs/bitcoin-regtest/sequencer_rollup_config.toml --sequencer
→ resources/configs/bitcoin-regtest/sequencer_config.toml --genesis-paths
→ resources/genesis/bitcoin-regtest/
```

- Mine 1 block in bitcoin (1 is enough because always reorg detected):
- `bitcoin-cli -regtest -rpcuser=citrea -rpcpassword=citrea -generate 1`.
- Sequencer's log:

```
2025-08-13T15:11:57.316281Z DEBUG sov_modules_stf_blueprint::stf_blueprint: Beginning l2 block #1842
→ from sequencer: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddb21325eff73f7
2025-08-13T15:11:57.332514Z INFO citrea_sequencer::runner: New block #1842, Tx count: #0
2025-08-13T15:11:57.332561Z DEBUG citrea_sequencer::runner: New L2 block with hash:
→ "a5fd813c2d14266d9e91a5fffc3ac3fad101c8b460424d67a20d2a38a8ac9c1"
2025-08-13T15:11:57.350581Z DEBUG L1BlockMonitor: bitcoin_da::service: Getting block with hash
→ 423e8587651f134097d028b869e06c2507792788a1b48c93abc0c72edcbaef1e
2025-08-13T15:11:57.351660Z DEBUG L1BlockMonitor: bitcoin_da::service: Getting block at height 333
2025-08-13T15:11:57.352148Z DEBUG L1BlockMonitor: bitcoin_da::service: Getting block with hash
→ 423e8587651f134097d028b869e06c2507792788a1b48c93abc0c72edcbaef1e
2025-08-13T15:11:57.353125Z DEBUG L1BlockMonitor: citrea_sequencer::da: Sequencer: last finalized L1
→ height: 333
2025-08-13T15:11:57.353153Z DEBUG L1BlockMonitor: bitcoin_da::fee: Fee rate: 1 sat/vb
-----reorg_detected
2025-08-13T15:11:58.313721Z DEBUG citrea_sequencer::runner: Saving short header proofs to ledger db
2025-08-13T15:11:58.316388Z DEBUG sov_modules_stf_blueprint::stf_blueprint: Beginning l2 block #1843
→ from sequencer: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddb21325eff73f7
2025-08-13T15:11:58.327823Z INFO citrea_sequencer::runner: New block #1843, Tx count: #0
2025-08-13T15:11:58.327867Z DEBUG citrea_sequencer::runner: New L2 block with hash:
→ "e7f7714373bf7e4485cd1c41ee0b1ee25c4f2e93cf697a5be2f828a3e7f4ea56"
2025-08-13T15:11:59.313062Z DEBUG citrea_sequencer::runner: Saving short header proofs to ledger db
2025-08-13T15:11:59.314485Z DEBUG sov_modules_stf_blueprint::stf_blueprint: Beginning l2 block #1844
→ from sequencer: 0x036360e856310ce5d294e8be33fc807077dc56ac80d95d9cd4ddb21325eff73f7
2025-08-13T15:11:59.323723Z INFO citrea_sequencer::runner: New block #1844, Tx count: #0
```

Recommendation: Compare at same depth: Instead of checking position in old array, compare blocks at same depth:

```
if let Some(&(stored_hash, stored_height)) = chain_state.recent_blocks.get(i - 1) {
    if current_hash == stored_hash && height == stored_height {
        break;
    } else {
        // Different block at this depth - reorg!
        reorg_detected = true;
        reorg_depth = i;
    }
}
```

Chainway Labs: Fixed in [citrea PR 2734](#).

Jakub: Fix verified.

3.2.4 Insufficient Funds Due to Incorrect Amount Comparison

Submitted by [Petrus](#)

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Summary: The issue is in the fn `choose_utxos`. After processing the required UTXO, `amount` is reduced by the required UTXO's value (`amount -= sum`). This updated `amount` correctly represents the remaining amount needed. Meaning: After subtracting the required UTXO's amount, `amount` holds the remaining balance needed.

However, in the smaller UTXOs loop, the condition checks `sum >= amount` using the original total amount (not the remaining amount), because `sum` includes the required UTXO's value while `amount` now only represents the gap. This implies that the smaller UTXOs loop incorrectly checks if `sum` (which includes the required UTXO + added UTXOs) covers this remaining amount (`sum >= amount`), rather than the original total amount.

Example of failure:

- Required UTXO: 40 ADA.
- Target amount: 100 ADA.
- Smaller UTXOs: [50 ADA].
- Logic:
 - After required UTXO: `sum = 40`, `amount = 60` (remaining).
 - Add 50 ADA → `sum = 90`.
 - Check `90 >= 60` → true (breaks loop).
 - Result: 90 ADA total (below target of 100).

Recommendation: The loop condition should track the remaining gap separately or compare against the original target. Currently, it underestimates the required total by ignoring that `sum` includes the required UTXO.

Chainway Labs: Fixed in [citrea PR 2693](#).

Jakub: Fix verified.

3.2.5 Infinite HTTP client timeout in FeeService can freeze the sequencer

Submitted by [marchev](#)

Severity: Medium Risk

Context: [fee.rs#L204](#)

Summary: The sequencer's `FeeService` fetches Bitcoin fee rates from the `mempool.space` API using a `request` HTTP client without a configured timeout. In this default state, `request` applies no time limit to the request, causing the client to wait indefinitely if the network call never completes. This can block the `FeeService` and freeze critical sequencer operations.

Description: The `FeeService` calls `get_fee_rate_from_mempool_space()`, which uses `request::get()` to query `mempool.space`'s API. This helper internally builds a new `request::Client`:

```
pub async fn get<T: IntoUrl>(url: T) -> crate::Result<Response> {
    Client::builder().build()?.get(url).send().await
}
```

In `request v0.12.12`, `Client::builder().build()` produces a `ClientBuilder` with the default configuration in which both `timeout` and `read_timeout` set to `None`:

```
pub fn new() -> Self {
    // ...
    ClientBuilder {
        config: Config {
            read_timeout: None, // Infinite read timeout
        }
    }
}
```

```

        timeout: None,      // Infinite total timeout
        // ...
    }
}

```

When `execute_request()` in `request::client` builds the `PendingRequest` object, these values are mapped into timer futures:

```

let total_timeout = timeout
    .or(self.inner.request_timeout)
    .map(tokio::time::sleep) // None -> None, Some -> Sleep
    .map(Box::pin);

let read_timeout_fut = self.inner.read_timeout
    .map(tokio::time::sleep) // None -> None, Some -> Sleep
    .map(Box::pin);

```

If both are `None`, no timers are created. In the `Future` implementation for `PendingRequest`, these timers are only polled if present:

```

if let Some(delay) = self.as_mut().total_timeout().as_mut().as_pin_mut() {
    // Only polled if Some()
    if let Poll::Ready(()) = delay.poll(cx) {
        return Poll::Ready(Err(...));
    }
}

if let Some(delay) = self.as_mut().read_timeout().as_mut().as_pin_mut() {
    // Only polled if Some()
    if let Poll::Ready(()) = delay.poll(cx) {
        return Poll::Ready(Err(...));
    }
}

```

With both of them absent, no timeout logic runs and the request will wait indefinitely if the TCP connection stalls or no data arrives. This could happen under normal network conditions (network partition, routing issues) or be caused deliberately if an attacker controls infrastructure (e.g., ISP, cloud provider) and performs a BGP hijacking or traffic blackholing attack.

This is a liveness issue because `get_fee_rate_from_mempool_space()` is awaited directly in critical sequencer logic:

- DA transaction queue - fee fetch blocks the loop, preventing new DA transactions from being sent. ([service.rs#L246-L259](#)).
- L1 block detection - `get_da_block_data()` depends on `get_fee_rate()`; if blocked, the `L1BlockMonitor` stops detecting new DA blocks. ([da.rs#L35](#)).
- Sequencer startup - `get_da_block_data()` is called in `run()` before the main loop; if blocked, the sequencer may never finish booting. ([runner.rs#L685-L692](#)).

Impact Explanation: This bug can fully halt the sequencer:

1. Commitments to the DA layer stop.
2. Detection of new L1/DA blocks ceases.
3. Startup can hang indefinitely.

No failover or timeout recovery means the sequencer remains stalled until manual intervention (e.g. restart).

Likelihood Explanation: The issue could occur:

- Under normal Internet conditions (e.g. network partition, routing black hole);
- During targeted infrastructure attacks (e.g. BGP hijacking, ISP-level blocking);

Proof of Concept: While the sequencer is running, drop outbound traffic to `mempool.space`:

```

sudo iptables -A OUTPUT -d mempool.space -j DROP

```

Then trigger or wait for any sequencer action that depends on `get_fee_rate()`. Observe that DA transactions and block detection halt. Restarting the sequencer will also fail during initialization.

Recommendation: Set explicit timeouts for the request HTTP client in the `FeeService`:

```
let client = request::Client::builder()
    .timeout(Duration::from_secs(5))
    .build()?;

let resp = client.get(url).send().await?;
```

Also, avoid fee rate fetching in contexts where it is unnecessary (e.g., DA block data fetch during startup).

Chainway Labs: Fixed in [citrea PR 2761](#).

Jakub: Fix verified.

3.2.6 Off-by-one error in `get_pending_commitments` causes chain halt

Submitted by [Goran](#), also found by [astrotimi](#) and [0x4lifemen](#)

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Summary: An off-by-one error in the `SequencerLedgerOps::get_pending_commitments` function causes the last pending commitment in a consecutive sequence to be omitted from retrieval, potentially leading to a complete halt of the L2 commitment pipeline.

Finding Description: The `get_pending_commitments` function in `SequencerLedgerOps` contains a critical off-by-one error when retrieving pending sequencer commitments:

```
fn get_pending_commitments(&self) -> anyhow::Result<Vec<SequencerCommitment>> {
    let commitment_indexes = self.db.get::<PendingSequencerCommitment>(&())?;
    match commitment_indexes {
        Some(mut commitment_indexes) => {
            if commitment_indexes.is_empty() {
                return Ok(vec![]);
            }
            commitment_indexes.sort_unstable();
            let start = commitment_indexes[0];
            let end = commitment_indexes[commitment_indexes.len() - 1];
            // @audit BUG: Exclusive range
            self.get_data_range::<SequencerCommitmentByIndex, _, _>(&(start..end))
        }
        None => Ok(vec![]),
    }
}
```

The implementation of `get_data_range` is as follows:

```
fn get_data_range<T, K, V>(&self, range: &std::ops::Range<K>) -> Result<Vec<V>, anyhow::Error>
where
    T: Schema<Key = K, Value = V>,
    K: Into<u64> + Copy + SeekKeyEncoder<T>,
{
    let mut raw_iter = self.db.iter()?;
    // @audit max_items will be one-too-small because of the incorrect range provided
    let max_items = (range.end.into() - range.start.into()) as usize;
    raw_iter.seek(&range.start)?;
    let iter = raw_iter.take(max_items);
    let mut out = Vec::with_capacity(max_items);
    for res in iter {
        let batch = res?.value;
        out.push(batch)
    }
    Ok(out)
}
```

The bug occurs because Rust ranges (`start..end`) (provided by `get_pending_commitments` to `get_data_range`) are exclusive of the end value. When consecutive pending commitments exist (e.g., `[5, 6, 7]`), the function retrieves commitments 5 and 6 (because `max_items` is calculated to be 2, when it should have been 3), missing commitment 7.

Exact same bug gets triggered also in a more likely case where there is a single pending commitment. Let's assume sequencer creates new pending commitment with index 7 and immediately crashes before submitting the Bitcoin TX. When `get_pending_commitments` is called by sequencer after it is restarted, it will return empty list when it should return list with a single element [7].

This breaks the commitment chain's integrity in two critical ways:

1. During sequencer restart: the `resubmit_pending_commitment` function uses this buggy method, causing the last pending commitment to remain unprocessed and not submitted to Bitcoin DA.
2. During commitment controller initialization: the `load_next_commitment_index_and_start_height` function also uses this method, potentially causing the sequencer to create duplicate commitment indices with different content.

When a commitment is indefinitely lost (ie. pending sequencer commitment 7 does not get submitted by sequencer due to the bug):

- Full node cannot finalize future commitments (each requires its predecessor to be finalized).
- The commitment chain halts at the missing commitment.
- L2 blocks after this point remain uncommitted.
- No new proofs can be generated (they require committed commitments).

Impact Explanation: High - can cause a complete halt of the L2 commitment and proof pipeline. Security guarantees of the rollup are compromised as L2 state transitions are no longer verifiable on L1.

Likelihood Explanation: Medium - the bug triggers when one or more consecutive commitments accumulate as pending, which occurs when sequencer crashes or restarts, or when DA service issues cause delays in Bitcoin transaction submission.

Proof of Concept: Here's a simple POC - in `CommitmentService`'s `commit` we add debug logs immediately after sequencer creates new `PendingSequencerCommitment` using `put_pending_commitment`:

```
ledger_db.put_pending_commitment(&commitment).map_err(|_| {
    anyhow!("Sequencer: Failed to store sequencer commitment in pending commitments")
})?;

+ debug!("debug-pending: Pending commitment stored: {:?}", commitment.index);
+ debug!("debug-pending: get_pending_commitments: {:?}", ledger_db.get_pending_commitments());
```

Now re-build the project, run the bitcoin chain:

```
bitcoind -regtest -txindex=1 -addresstype=bech32m -fallbackfee=0.0001 -rpcport=18443 -rpcuser=citrea
↪ -rpcpassword=citrea
bitcoin-cli -regtest -rpcuser=citrea -rpcpassword=citrea loadwallet citretesting
bitcoin-cli -regtest -rpcuser=citrea -rpcpassword=citrea -generate 201
```

Start sequencer:

```
CITREA_VERBOSEITY=1 ./target/debug/citrea --dev --da-layer bitcoin --rollup-config-path
↪ resources/configs/bitcoin-regtest/sequencer_rollup_config.toml --sequencer
↪ resources/configs/bitcoin-regtest/sequencer_config.toml --genesis-paths resources/genesis/bitcoin-regtest
```

Now, the expectation is that debug statements will show the pending commitment which was just created. However, due to the bug `get_pending_commitments` returns empty list instead of the list with a single element:

```
cat logs/sequencer.log | grep "debug-pending"
2025-08-09T13:47:09.707901Z DEBUG CommitmentService: citrea_sequencer::commitment::service: debug-pending:
↪ Pending commitment stored: 1
2025-08-09T13:47:09.707992Z DEBUG CommitmentService: citrea_sequencer::commitment::service: debug-pending:
↪ get_pending_commitments: Ok([])
2025-08-09T13:48:49.707621Z DEBUG CommitmentService: citrea_sequencer::commitment::service: debug-pending:
↪ Pending commitment stored: 2
2025-08-09T13:48:49.707696Z DEBUG CommitmentService: citrea_sequencer::commitment::service: debug-pending:
↪ get_pending_commitments: Ok([])
2025-08-09T13:50:29.708950Z DEBUG CommitmentService: citrea_sequencer::commitment::service: debug-pending:
↪ Pending commitment stored: 3
2025-08-09T13:50:29.709185Z DEBUG CommitmentService: citrea_sequencer::commitment::service: debug-pending:
↪ get_pending_commitments: Ok([])
```

Recommendation: Fix the off-by-one error:

```
fn get_pending_commitments(&self) -> anyhow::Result<Vec<SequencerCommitment>> {
    let commitment_indexes = self.db.get:::<PendingSequencerCommitment>(&())?;
    match commitment_indexes {
        Some(mut commitment_indexes) => {
            if commitment_indexes.is_empty() {
                return Ok(vec![]);
            }
            commitment_indexes.sort_unstable();
            let start = commitment_indexes[0];
            - let end = commitment_indexes[commitment_indexes.len() - 1];
            + let end = commitment_indexes[commitment_indexes.len() - 1] + 1;
            self.get_data_range:::<SequencerCommitmentByIndex, _, _>(&(start..end))
        }
        None => Ok(vec![]),
    }
}
```

Chainway Labs: Fixed in [citrea PR 2787](#). We removed the "pending" commitment feature from the codebase, as it was redundant and not needed anymore.

Jakub: Do note that the change in the provided PR is quite big, and related to the entire pending commitment logic. However it looks good to me.

3.2.7 Bitcoin DA Unbounded pre-decompression accumulation causes DoS/OOM

Submitted by [hecok](#)

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Summary: The Bitcoin DA service concatenates an arbitrary number of compressed chunks into a single `Vec<u8>` prior to calling the decompressor. There is no effective guard on:

- Total compressed input size, and...
- Number of chunks referenced by an Aggregate.

As a result, memory usage can grow unbounded during block processing, leading to out-of-memory and node crash (DoS) before the decompressor's limit check on the decompressed output is ever reached.

Finding Description:

1. Affected path: `crates/bitcoin-da/src/service.rs::extract_relevant_zk_proofs()`.
2. Affected function: `service.rs::extract_relevant_zk_proofs()`.
3. When a signed Aggregate transaction is processed, the service:
 - Decodes the list of `chunk_ids` from `aggregate.body`.
 - Fetches each referenced chunk and appends its compressed bytes to a single `Vec<u8>`.
 - Calls `decompress_blob(&body)` after the loop.

Because there is no limit on the number of chunks or on the cumulative compressed size, a prover can trigger unbounded memory allocation and crash the node (OOM) *before* decompression limits apply.

4. Contrast with decompressor implementation:
 - `crates/primitives/src/compression.rs::decompress_blob()` only enforces `MAX_DECOMPRESSED_BLOB_SIZE` on the decompressed output. It does not limit `blob.len()` (compressed input size), so OOM can occur during `body.extend(...)` before decompression runs.
5. Affected Code:
 - Aggregate signature/key verification gate
 - Allocation of accumulation buffer `let mut body = Vec::new();`
 - Parse chunk and append compressed bytes `body.extend(chunk)`

- Call to `decompress_blob(&body)` after accumulation
- Guard on decompressed output size (`MAX_DECOMPRESSED_BLOB_SIZE`)

Impact Explanation:

1. A block containing an Aggregate with many chunk transactions can cause the node to allocate large contiguous buffers and run out of memory. The crash happens before any logical validation outcome is produced, resulting in denial-of-service for the DA service and downstream rollup services.
2. This path is restricted to Aggregates signed by the authorized prover key; therefore, the DoS depends on a privileged/operational actor (self-DoS).

Likelihood Explanation:

1. Aggregate processing only proceeds when `aggregate.public_key() == prover_da_pub_key` and signature verification succeeds (`get_sig_verified_hash().is_some()`).
2. A misconfigured or evolving prover can accidentally produce very large Aggregates (many chunks / large compressed size), which will deterministically trigger the unbounded pre-decompression allocation on every node processing that block.

Proof of Concept:

- Behavior: appends compressed chunks into a single buffer (mirroring `service.rs` accumulation). Observe memory growth up to OOM prior to decompression.

Notes:

- In realistic conditions, `CHUNK_COMPRESSED_SIZE` will vary and the total accumulated size is a function of both per-chunk size and number of chunks. Neither is bounded today on the Aggregate path.

Recommendation: Enforce pre-decompression budgets in `service.rs`:

```
@@ async fn extract_relevant_zk_proofs(...)
-   let mut body = Vec::new();
+   let mut body = Vec::new();
+   const MAX_AGGREGATE_CHUNKS: usize = 4096;
+   const MAX_COMPRESSED_BLOB_SIZE: usize = 64 * 1024 * 1024; // 64MB
+   let mut chunk_count: usize = 0;
+   let mut total_compressed_bytes: usize = 0;
  @@ match parsed {
      let DataOnDa::Chunk(chunk) = data else {
          warn!("{tx_id}: Chunk: unexpected kind");
          continue 'aggregate;
      };
      chunk_count += 1;
      total_compressed_bytes += chunk.len();
      if chunk_count > MAX_AGGREGATE_CHUNKS
      || total_compressed_bytes > MAX_COMPRESSED_BLOB_SIZE
      {
          warn!(
              "{tx_id}: Aggregate budgets exceeded: chunks={}, compressed={} bytes",
              tx_id, chunk_count, total_compressed_bytes
          );
          continue 'aggregate;
      }
      body.extend(chunk);
```

Guard compressed input length in `compression.rs`:

```
@@ use std::cmp::min;
use std::io::{self, Read, Write};

- use crate::MAX_DECOMPRESSED_BLOB_SIZE;
+ use crate::MAX_DECOMPRESSED_BLOB_SIZE;
+ pub const MAX_COMPRESSED_BLOB_SIZE: usize = 64 * 1024 * 1024; // 64MB (tune per network)
  @@ pub fn decompress_blob(blob: &[u8]) -> Result<Vec<u8>, io::Error> {
      let mut reader = brotli::Decompressor::new(
+         if blob.len() > MAX_COMPRESSED_BLOB_SIZE {
+             return Err(io::Error::new(
+                 io::ErrorKind::InvalidData,
+                 "Compressed data exceeds maximum allowed size",
+             ));
+         }
      )
```

```
+ let mut reader = brotli::Decompressor::new(
    blob,
    4 * 1024, // 4kb buffer size
);
```

Chainway Labs: Fixed in [citrea PR 2768](#).

Jakub: Fix verified.

3.2.8 Unauthenticated DoS via JsTracer on debug_traceBlockByNumber Endpoint

Submitted by [franfran](#)

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Summary: The publicly exposed `debug_traceBlockByNumber` RPC endpoint allows arbitrary JavaScript execution through the JsTracer tracer option, enabling trivial denial-of-service attacks that can crash full nodes.

Finding Description: The `debug_traceBlockByNumber` endpoint accepts a `GethDebugTracingOptions` parameter containing a `tracer` field that can be set to `JsTracer(code)`. This JavaScript code is executed server-side using the Boa engine during EVM transaction tracing. While Boa provides loop iteration limits (200k) and recursion limits (10k), it lacks memory limits, allowing attackers to create memory-intensive operations that consume gigabytes of RAM per request. Since the endpoint is publicly accessible without authentication on full nodes, any external user can send malicious JavaScript payloads that execute expensive operations during each EVM step, leading to resource exhaustion and node crashes.

Impact Explanation: This is a High impact because it enables trivial denial-of-service attacks against full nodes. A single malicious request can consume multiple gigabytes of memory and crash the node, while multiple concurrent requests can completely overwhelm system resources. This directly breaks the availability guarantee of the RPC service and can render the entire node infrastructure unusable.

Likelihood Explanation: This vulnerability has High likelihood because the attack vector is trivially exploitable. The endpoint is publicly accessible without authentication, requires no special permissions or complex setup, and can be triggered with a simple HTTP POST request containing malicious JavaScript code.

Proof of Concept: Let's launch the infra:

```
# launch the sequencer
./target/debug/citrea --dev --da-layer mock --rollup-config-path
↳ resources/configs/mock/sequencer_rollup_config.toml --sequencer
↳ resources/configs/mock/sequencer_config.toml --genesis-paths resources/genesis/mock/
```

```
# launch the full node
./target/debug/citrea --dev --da-layer mock --rollup-config-path resources/configs/mock/rollup_config.toml
↳ --genesis-paths resources/genesis/mock/
```

We need to create a "step function" according to revm tracers, so let's just create a basic smart contract, deploy it, and call it. This step function is called for each opcode.

```
mkdir /tmp/foundry_project
cd /tmp/foundry_project
forge init --no-git
forge create src/Counter.sol:Counter -r http://127.0.0.1:12346 --private-key
↳ 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80 --broadcast
cast send 0x5FbDB2315678afecb367f032d93F642f64180aa3 "setNumber(uint256)" 123456789 -r http://127.0.0.1:12346
↳ --private-key 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
```

Now, extract the block number from the receipt. This is the one we are going to simulate. Here the block number is 0x23 (in hex). We want to run this malicious code on the full node machine:

```
let arr = [];
for(let i = 0; i < 200000; i++) {
    arr.push(new Array(10000).fill("x".repeat(1000)));
}
```

This will allow us to run a massive amount of parallel requests:

```
for i in {1..10000}; do curl -s -H "Content-Type: application/json" -X POST --data
↳ '{"jsonrpc":"2.0","method":"debug_traceBlockByNumber","params":["0x23"],{"tracer":{"step: function(log) { let
↳ arr = []; for(let i = 0; i < 2000000; i++) { arr.push(new Array(100000).fill(\\"x\\").repeat(10000)); } }},
↳ result: function() { return {}; }, fault: function() {}}}"]',"id":"' $i '}' 127.0.0.1:12346 & done; wait
```

From the first request, the full node operation has already halted because it's too concerned about allocating a lot of heap data. It doesn't produce any info log anymore:

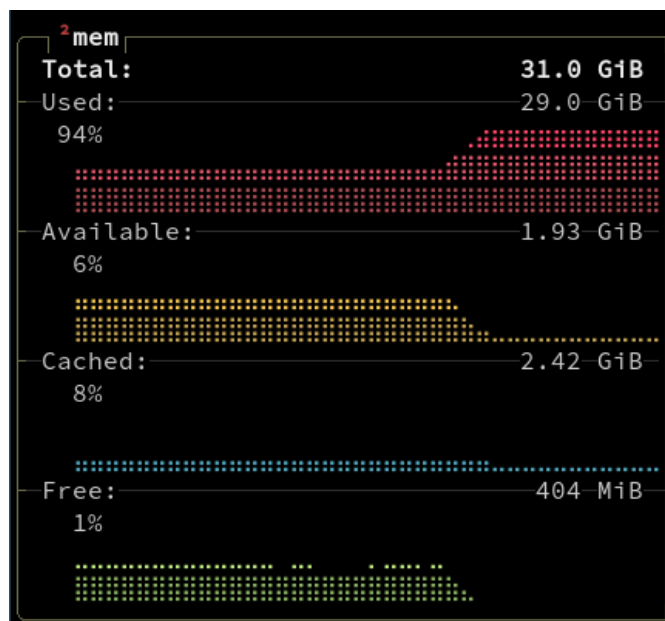
```
2025-08-07T05:46:50.863511Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #44 with hash:
↳ 0x0fba98cdc773824961c82d980a7343e0b4e6a06c442be52f12047cf3cc224d8d
2025-08-07T05:46:50.890564Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #44 is:
↳ 0x74589110487dfe6fd1564bb0610e4ddc5cf4bf49bae15d70ca24e60f58b2c9fa
2025-08-07T05:46:51.894577Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #45 with hash:
↳ 0x23a2f07ca96d0216a1c775e44befe8389349a57cf3fa3e9f6af05868b67adc281
2025-08-07T05:46:51.910733Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #45 is:
↳ 0x8ebaeceb958458142c34024b596bf0baf2e9c5d02673bae94fe1b4a558d9446a
2025-08-07T05:46:52.915347Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #46 with hash:
↳ 0xd3585c7d5877fa81e7b206e41a5a8b637b34378361e63d31f38f5574e8da7848
2025-08-07T05:46:52.936139Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #46 is:
↳ 0x7dfee6612d748f84a267f1ae99ab3eab2f40b70bf32cc01476df39ca627c4c4
2025-08-07T05:46:53.941956Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #47 with hash:
↳ 0x9f9c09547e8af5a26290bc3946d71989310e959f1a30be482852192dab8d74c1
2025-08-07T05:46:53.963843Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #47 is:
↳ 0x0a63467293eb186c07cdaa383da6288006d127207f1785726e156
2025-08-07T05:46:54.970611Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #48 with hash:
↳ 0x2f2ba83c1b95d19ce1c33e32b681301c3b2aff55d9f85a6930535f1ed92eccb1
2025-08-07T05:46:54.998976Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #48 is:
↳ 0x5904f0b70d104977848ad9462b76c4bc46b2735a9e30c87c2b0ba944f75b8f1f
2025-08-07T05:46:56.006001Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #49 with hash:
↳ 0xe2cca1bfdb06c213f6bdaf6e0ccc13ed5de982e6a2fa1fd388d20d2c68ebc
2025-08-07T05:46:56.029093Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #49 is:
↳ 0x26efff69b56334b649dabf24dc4534240f583440f95b6781ba3ed05934658f97
2025-08-07T05:46:57.036091Z INFO L2Syncer: citrea_common::l2: Running 12 block batch #50 with hash:
↳ 0xecd0c4332eb4c56e52955b13d8ddd7edefc6d5559bb20334a70ec775b64a0dbb
2025-08-07T05:46:57.057387Z INFO L2Syncer: citrea_common::l2: New State Root after 12 block #50 is:
↳ 0xd14f3148898f28067a675aac76b2b9e9fc64c9e65051031be67d486794246328
```

The node is irresponsive to any rpc call:

```
cast block-number -r http://127.0.0.1:12346
Error: error sending request for url (http://127.0.0.1:12346/)
```

```
Context:
- operation timed out
```

Using `btop`, we can also see that the memory usage spiked. We caused a memory leak and it's not taking long at all before seeing a crash!



Recommendation: Disable the JsTracer tracer entirely in production environments. Potentially disable all the debug_* endpoints too. [According to the geth docs](#):

The default whitelist allows access to the eth, net and web3 namespaces. To enable access to other APIs like debugging (debug), they must be configured using the --http.api flag. Enabling these APIs over HTTP is not recommended because access to these methods increases the attack surface.

Chainway Labs: Fixed in [citrea PR 2667](#). timeout wasn't possible so we went with option to enable/disable js tracers.

Jakub: Fix verified.

3.2.9 Failed deposit transactions cause nonce mismatch and break L2 block production

Submitted by [marchev](#)

Severity: Medium Risk

Context: [runner.rs#L1243-L1245](#)

Summary: If a deposit transaction reverts during block execution and is followed by other system transactions in the same block, the current L2 block production will fail. This happens because the nonce used for signing system transactions is not adjusted when a deposit fails, leading to a mismatch that causes all subsequent system transactions to be rejected.

Finding Description: Citrea handles deposit transactions as system transactions. During block production, each one is signed and assigned a nonce from a local counter:

```
let signed_tx = self.sign_tx(l2_block_info.current_spec, raw_message, *nonce)?;
*nonce += 1;
```

If a deposit transaction reverts (e.g. when the recipient is a contract that does not that does not accept cBTC), the working set is reverted:

```
if matches!(e, EvmSystemTransactionNotSuccessful) && is_deposit {
    working_set_to_discard = working_set.revert().to_revertable();
    continue; // <- nonce not decremented
}
```

The problem is that the local nonce is not decremented. So the next system transaction gets signed with a nonce that does not match the account's state. This causes the following validation in `pre_dispatch_tx_hook()` to fail:

```
if sender.nonce != tx_nonce {
    return Err(L2BlockHookError::SovTxBadNonce);
}
```

That error then bubble up and causes the entire current L2 block production to fail. Regular transactions do decrement the nonce on failure in `dry_run_transactions()`:

```
if let Err(e) = self
    .stf
    .apply_l2_block_txs(&l2_block_info, &txs, &mut working_set)
{
    // Decrement nonce if the transaction failed
    nonce -= 1;
    // ...
}
```

Whereas system transactions do not which creates the bug. There is a safeguard in `send_raw_deposit_transaction()` that simulates deposit transactions using `eth_call` before adding them to the mempool. This catches obviously invalid deposits. But there are two ways an attacker can bypass this protection:

- Attack vector 1: Stateless simulation.

The validation during simulation uses a clean working set for each deposit (`WorkingSet::new(self.context.storage.clone())`). This behavior explainably ignores earlier deposits in the deposit mempool.

Thus, an attacker could: 1. Submit the same valid deposit 3 times quickly. 2. Each passes simulation because they use the same snapshot. 3. During block execution, the first succeeds. 4. The second fails because it reuses the same move transaction. The failed second deposit causes the nonce mismatch. 6. The third deposit is rejected due to the mismatched nonce. 7. The error fails the current L2 block production.

- Attack vector 2: Timestamp mismatch in simulation.

Deposit transactions in `send_raw_deposit_transaction()` are simulated by running an `eth_call` against the pending state:

```
let tx_res = evm.get_call(  
  dep_tx,  
  Some(BlockId::pending()), // <- pending block state  
  None,  
  None,  
  &mut working_set,  
  &self.context.ledger,  
);
```

in `Evm::get_call_inner()` the `BlockId::pending()` variant is internally mapped to a simulation environment through `get_pending_block_env()`:

```
BlockId::Pending => {  
  let env = get_pending_block_env(&ledger);  
  // ...  
  env  
}
```

This function builds the block environment used during simulation by copying the latest block's header and incrementing the block number and adjusting other block context properties as well. However, it does not update the block timestamp. The simulation ends up using the stale timestamp from the latest block instead of the current system time.

This creates a predictable and exploitable divergence between simulation and actual execution:

- During simulation: `timestamp = latest_block.timestamp` (stale).
- During execution: `timestamp = now()` (a few seconds later).

This allows an attacker to craft a contract that behaves differently and receive or reject incoming cBTC based on `block.timestamp`. For example:

1. Deploy a malicious contract whose `receive()` function only accepts cBTC transfers if `block.timestamp` equals a specific value (e.g. latest block's timestamp).
2. Submit two deposit transactions targeting that contract.
3. Both pass simulation since they are validated against the same stale timestamp.
4. When the block is actually executed during its dry run, the current timestamp causes the contract to revert the first deposit. This failed deposit triggers the nonce mismatch bug.
5. The second deposit is then rejected due to the bad nonce.
6. As before, the block build fails.

Describe which security guarantees it breaks and how it breaks them. If this bug does not automatically happen, showcase how a malicious input would propagate through the system to the part of the code where the issue occurs.

Impact Explanation: If a deposit fails in this way, the entire L2 block fails to build. All deposit transactions selected for that block are lost, as they are removed from the mempool during processing.

Likelihood Explanation: Accidental failure is unlikely thanks to the simulation check. But an attacker can bypass it using the above described attack vectors.

Proof of Concept: See detailed steps of the possible attack vectors above at the end of the Finding Description section.

Recommendation: Update `process_sys_txs()` to decrement the nonce when a deposit transaction fails. For example:

```

if matches!(e, EvmSystemTransactionNotSuccessful) && is_deposit {
    *nonce -= 1; // <- keep local nonce in sync
    working_set_to_discard = working_set.revert().to_revertable();
    continue;
}

```

Chainway Labs: Fixed in [citrea PR 2713](#). We won't decrement nonce on failing non-deposit system tx, because that means there is a bug in our implementation that needs fixing.

Jakub: Fix verified.

3.2.10 Proof-Slot Leak Halts Parallel Prover

Submitted by [AlmanaxAI](#)

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Summary: `ParallelProverService::start_proving` increments the global "ongoing-proof" counter before it knows a zk-VM job has started. If `make_proof` returns an error, the function exits early without decrementing the counter or notifying waiters. Each failure permanently consumes one of the limited parallel slots; after `PARALLEL_PROOF_LIMIT` such errors the prover stops accepting new jobs and roll-up proofs cease.

Finding Description: File `crates/prover-services/src/parallel.rs`:

```

self.reserve_proof_slot().await; // +1

let proof_rx = make_proof(vm, ...).context("Failed to start proving"); // + ? early-return on Err

```

The only decrement lives inside the task that is spawned after `make_proof` succeeds:

```

tokio::spawn(async move {
    let proof = proof_rx.await;
    *ongoing_proof_count.lock().await -= 1; // -1
    notifier.notify_one();
});

```

Error path: if `make_proof` fails (bad ELF, guest panic, I/O error) the `?` propagates, the task is never spawned, the counter is never decremented, and `proof_done_notifier` is never signalled:

- Git grep shows no other code writes to `ongoing_proof_count`.
- `make_proof` propagates any `vm.run()` failure; the zkVM returns `Err` for invalid circuit input, corrupted binaries, or internal panics-all realistic in production when DA data is malformed.

Impact and Likelihood Explanation: Impact High: once the leak happens `reserve_proof_slot` blocks every subsequent call; batch and light-client provers stop generating proofs, holding withdrawals and bridge finality hostage.

Likelihood Medium: failures in `make_proof` arise from ordinary operator mistakes (wrong ELF), disk corruption, or unforeseen guest panics; no attacker input is required:

- After `PARALLEL_PROOF_LIMIT` (default 4-16) consecutive setup errors the service is irrevocably stuck until a manual restart.

Proof of Concept: Running the test with `cargo test -p prover-services slot_leak_dos -- --nocapture` shows the third call never returns, proving the denial of service.

```

#[tokio::test]
async fn slot_leak_dos() {
    use prover_services::{parallel::ParallelProverService, ProofData, ProofGenMode};
    use std::sync::Arc;

    let bad_elf = vec![0u8; 16]; // corrupt ELF + make_proof fails
    let dummy_da = Arc::new(MockDa {});
    let prover = ParallelProverService::new(
        dummy_da,
        DummyVm::default(),
        ProofGenMode::ProveWithSampling,
    );
}

```

```

    2,          // PARALLEL_PROOF_LIMIT = 2
)
.unwrap();

let data = ProofData { input: vec![], assumptions: vec![], elf: bad_elf };

// First failure leaks slot 1
assert!(prover.start_proving(data.clone(), ReceiptType::Groth16, Uuid::nil()).await.is_err());
// Second failure leaks slot 2
assert!(prover.start_proving(data, ReceiptType::Groth16, Uuid::nil()).await.is_err());

// Third call hangs forever (test times out) because all slots are leaked
}

```

Recommendation: Move slot reservation after make_proof succeeds, or add a guard:

```

let proof_rx = make_proof(vm, job_id, elf, self.proof_mode, receipt_type)
    .context("Failed to start proving")?;

self.reserve_proof_slot().await;          // increment only now

```

or

```

self.reserve_proof_slot().await;
let proof_rx = match make_proof(..) {
    Ok(rx) => rx,
    Err(e) => {
        *self.ongoing_proof_count.lock().await -= 1;
        self.proof_done_notifier.notify_one();
        return Err(e);
    }
};

```

Either fix guarantees slot accounting remains correct and the prover service cannot dead-lock.

Chainway Labs: Fixed in [citrea PR 2702](#).

Jakub: Fix verified.